

Logos Runtime vs Compile Time Costs

<https://gemini.google.com/app/450de56dc3a49313>

User prompt: How would you combine winnow with memchr or memchr?

Response: Combining `winnow` with `memchr` / `memmem` is straightforward because `winnow` is built to work natively on `&str` and `[u8]` slices.

Instead of writing a `winnow` combinator that tests bytes individually inside a predicate closure, you use `memchr` or `memmem` to **instantly find the byte index of your boundary**, split the slice at that index, and advance `winnow`'s internal pointer.

Here is how you wire them together cleanly.

1. Extending `winnow` with Custom `memchr` Parsers

In `winnow`, a parser is just a function that takes `&mut Input` and returns a `PResult<Output>`.

You can write custom, reusable helper functions (or extension traits) that invoke `memchr` directly on the input slice:

```
use winnow::prelude::*;
use winnow::error::ContextError;
use memchr::memchr;

/// Parses plain text up to the next occurrence of `delimiter` byte (e.g., `\" or `|`).
/// Uses SIMD-accelerated `memchr` under the hood.
pub fn take_until_byte<'s>(
    delimiter: u8
) -> impl FnMut(&mut &'s str) -> PResult<&'s str, ContextError> {
    move |input: &mut &'s str| {
        // Access raw bytes without copying
        let bytes = input.as_bytes();

        match memchr(delimiter, bytes) {
            Some(idx) => {
                // Split the string slice at the SIMD-located boundary
                let (head, tail) = input.split_at(idx);
                *input = tail; // Advance winnow's input reference
            }
            None => {
                // No delimiter found, consume the rest of the input
                *input = &input[input.len()..];
            }
        }
    }
}
```

```

        Ok(head)
    }
    None => {
        // If delimiter isn't found, consume the remaining slice entirely
        let head = *input;
        *input = "";
        Ok(head)
    }
}
}
}
}

```

2. Multi-Byte Sequences with memmem

For USFM structural boundaries like end-markers (`\f*`, `\+w*`) or chapter markers (`\c`), you use `memmem::Finder`:

```

use memchr::memmem::Finder;
use winnow::prelude::*;
use winnow::error::ContextError;

/// Takes text up to a multi-byte sequence like `\f*` using SIMD `memmem`.
pub fn take_until_sequence<'s>(
    finder: &'static Finder<'static>
) -> impl FnMut(&mut &'s str) -> PResult<&'s str, ContextError> {
    move |input: &mut &'s str| {
        let bytes = input.as_bytes();

        match finder.find(bytes) {
            Some(idx) => {
                let (head, tail) = input.split_at(idx);
                *input = tail;
                Ok(head)
            }
            None => {
                let head = *input;
                *input = "";
                Ok(head)
            }
        }
    }
}

```

```

    }
  }
}

```

3. Putting It Together: A Single-Pass USFM Fragment Parser

Now you can mix standard `winnow` combinators (for small, precise tokens like `\v 12`) with `memchr` / `memmem` (for sweeping over hundreds of bytes of verse text at 10+ GB/s):

```

use winnow::prelude::*;
use winnow::token::take_while;
use winnow::ascii::digit1;
use memchr::memchr2;
use std::sync::LazyLock;
use memchr::memmem::Finder;

static FOOTNOTE_END: LazyLock<Finder<'static>> = LazyLock::new(|| Finder::new(b"\\f*"));

#[derive(Debug, PartialEq)]
pub enum UsfmNode<'a> {
    VerseNumber(&'a str),
    Text(&'a str),
    Footnote(&'a str),
}

// 1. Precise Winnow parser for \v 12
fn parse_verse_marker<'s>(input: &mut &'s str) -> PResult<&'s str> {
    "\\v ".parse_next(input)?;
    let num = digit1.parse_next(input)?;
    let _ = " ".parse_next(input); // Optional trailing delimiter space
    Ok(num)
}

// 2. SIMD-accelerated text sweep up to the next \ or |
fn parse_plain_text<'s>(input: &mut &'s str) -> PResult<&'s str> {
    let bytes = input.as_bytes();

```

```

// memchr2 searches for either '\\\'' OR '|' simultaneously in SIMD registers
match memchr2(b'\\\'', b'|', bytes) {
    Some(0) => Err(winnow::error::ErrMode::Backtrack(
        winnow::error::ContextError::default(),
    )),
    Some(idx) => {
        let (head, tail) = input.split_at(idx);
        *input = tail;
        Ok(head)
    }
    None if !input.is_empty() => {
        let head = *input;
        *input = "";
        Ok(head)
    }
    None => Err(winnow::error::ErrMode::Backtrack(
        winnow::error::ContextError::default(),
    )),
}
}

```

```

// 3. Fast Footnote parser switching to memmem finder
fn parse_footnote<'s>(input: &mut &'s str) -> PResult<&'s str> {
    "\\f ".parse_next(input)?;
    let content = take_until_sequence(&FOOTNOTE_END).parse_next(input)?;
    "\\f*".parse_next(input)?;
    Ok(content)
}

```

```

// Master parser dispatch
pub fn parse_usfm_node<'s>(input: &mut &'s str) -> PResult<UsfmNode<'s>> {
    alt((
        parse_verse_marker.map(UsfmNode::VerseNumber),
        parse_footnote.map(UsfmNode::Footnote),
        parse_plain_text.map(UsfmNode::Text),
    ))
}

```

```
   )).parse_next(input)
}
```

Why This Combination Is so Effective

1. **Zero Allocations:** Everything remains a `&'a str` slice of the original memory buffer.
 2. **Optimal Branching:** `winnow` handles structural matching and error backtracking near markers, while `memchr` skips whole blocks of text without running character-by-character loops.
 3. **No Intermediate Token Buffer:** You generate your AST nodes (e.g. `UsfmNode::Text`) on the fly in a single pass.
-

User prompt: You said threading parser state through with winnow was finicky though?

Response: Ah, that's my fault for not being clearer earlier—let's untangle that!

Threading state in `winnow` isn't *impossible*, but if you try to do it using `winnow`'s built-in state mechanism (`Stateful<I, S>`), it can feel **finicky and type-heavy** compared to simple imperative Rust.

Here is why it gets awkward, and how to cleanly bypass the finickiness if you need parser state.

Why `winnow`'s Built-In `Stateful` Feels Finicky

`winnow` provides a wrapper called `Stateful<Input, State>`. The friction comes from two places:

1. Borrow Checker Fights inside Closures:

Because `winnow` combinators take `&mut Input`, when `Input` is a `Stateful<&str, MyState>`, accessing both the string slice and updating your state at the same time requires splitting borrows on the wrapper struct. In deeply nested combinator closures (`alt`, `dispatch`, `repeat`), Rust's borrow checker will frequently complain about partial moves or mutable references overlapping.

2. Monadic Type Pollution:

As soon as a helper parser expects a `Stateful` input, its function signature changes. You can no longer easily compose a plain `fn(&mut &str) -> PResult<&str>` with a stateful parser without constantly wrapping and unwrapping the state

context.

How to Keep State Clean (3 Approaches)

Depending on what kind of state you actually need for USFM, here is how you avoid the friction:

Approach 1: The "Context-Free First Pass" (Recommended for USFM)

For 90% of USFM parsing, **you don't actually need mutable parser state during the slice phase.**

Instead, construct a clean, stateless AST or token stream in pass 1, and then resolve contextual state (like matching milestone opening/closing pairs or tracking unclosed character markers) in a fast, sequential pass over the AST.

```
// Pass 1: Stateless Winnow + memchr pass -> UsfmNode
// Pass 2: Simple imperative loop to track context
struct UsfmContext {
    current_chapter: u32,
    open_markers: Vec<String>,
}

fn validate_and_resolve(nodes: Vec<UsfmNode>) {
    let mut ctx = UsfmContext::default();
    for node in nodes {
        match node {
            UsfmNode::Marker("c") => { /* update ctx */ },
            UsfmNode::MilestoneStart(id) => { ctx.open_markers.push(id); },
            // ...
        }
    }
}
```

Approach 2: Method-Based Struct Parser (Zero Combinator Finickiness)

Instead of using `winnow`'s monadic `Stateful` wrapper or passing `&mut Context` through pure combinators, wrap the input slice AND state inside your own Parser struct, and write standard Rust methods:

```

pub struct UsfmParser<'a> {
    pub input: &'a str,
    pub current_chapter: u32,
    pub in_footnote: bool,
}

impl<'a> UsfmParser<'a> {
    // A method that uses winnow internally, but has direct &mut self access!
    pub fn parse_next_node(&mut self) -> PResult<UsfmNode<'a>> {
        if self.in_footnote {
            self.parse_footnote_content()
        } else {
            // Mutate self.input via winnow while freely reading/mutating self.current_chapter
            parse_verse_marker.parse_next(&mut self.input)?;
            Ok(UsfmNode::....)
        }
    }
}

```

Why this works: You get full `&mut self` access to your custom state fields without fighting closures, generic trait bounds, or wrapper types.

Approach 3: Threading Explicit `&mut` State as a Parameter

If you *do* want standalone functions, skip `winnow::Stateful` and just pass your state explicitly alongside the input reference:

```

fn parse_marker<'s>(
    input: &mut &'s str,
    state: &mut ParserState
) -> PResult<UsfmNode<'s>> {
    let name = parse_marker_name.parse_next(input)?;
    state.last_marker = Some(name); // Clean, standard Rust mutation
    Ok(UsfmNode::Marker(name))
}

```

Bottom Line

The "finicky" reputation comes from trying to force `winnow` to manage state through its generic functional combinators.

If you use **Winnow purely for slice advancement** (`&mut &str`) and use standard **Rust struct methods or explicit `&mut State` args** for state, it stays fast, ergonomic, and completely painless.

User prompt: Ok, and given the constraints of isfm, ie unaligned text are usually just like \c 1 \v 1 lot or text til end of chapter, Some interspersed poetry (most field texts are intentionally limited), how do you leverage that it's ascii delimiters, and only the \ and optbreak of // and then attributes (but that's out of this scope) to lever mage the domain with wither memmem or memchr or swar or just to be as speedy as possible

Response: To squeeze absolute maximum performance out of USFM/ISFM, you exploit one core truth: **90%+ of the file is dumb byte payload**. You shouldn't be running a state machine or parsing logic on verse text at all.

Because USFM relies on fixed ASCII delimiters (`\` and `//`) and has a tiny, predictable marker set, you can build a parser that runs at **memory-bandwidth speeds (10–20+ GB/s)** by combining three hyper-specific techniques:

1. **SIMD Multi-Byte Sweeping (`memchr2`)** to skip text payloads in multi-gigabyte chunks.
2. **SWAR / u32 Register Casting** to identify markers in a single CPU instruction (zero string matching).
3. **Branchless ASCII Number Parsing** for verse/chapter numbers.

Here is the exact architectural blueprint to maximize these domain constraints.

Technique 1: Dual-Target SIMD Scanning (`memchr2`)

Instead of inspecting every byte for boundaries, use `memchr2` to scan hardware vector registers simultaneously for both target ASCII triggers: `\` (`0x5C`) and `/` (`0x2F`).

```
use memchr::memchr2;
```

```
// memchr2 checks 16 to 64 bytes PER CPU CYCLE for either '\ ' or '/'
while let Some(hit_idx) = memchr2(b'\\', b'/', rest.as_bytes()) {
    // 1. Instantly slice and yield everything before `hit_idx` as plain verse text
    let (text, tail) = rest.split_at(hit_idx);
    if !text.is_empty() {
```

```

    emit_text(text);
}

// 2. Dispatch based on the trigger character
if tail.as_bytes()[0] == b'\\' {
    rest = parse_backslash_marker(tail);
} else {
    // Handle optional '/' line breaks
    rest = parse_slash_break(tail);
}
}

```

- **Why it's so fast:** If a verse is 150 characters long, `memchr2` skips across it in **3–4 CPU clock cycles**, completely ignoring all normal words, punctuation, and spaces.

Technique 2: SWAR / u32 Integer Marker Dispatch

Standard lexers extract string slices (like `"v"`, `"c"`, `"q1"`) and compare them against string tables. That requires boundary checks and hashing.

Because USFM markers are tiny ASCII strings followed by a space or newline, **you can read the 2 to 4 bytes immediately following \ directly into a standard 32-bit register (u32).**

Instead of comparing strings, your marker match compiles down to a single numeric `match` or hardware jump table:

```

#[inline(always)]
fn parse_backslash_marker<'a>(input: &'a str) -> &'a str {
    let bytes = input.as_bytes();
    // Safety guard: ensure we have at least 4 bytes to read a u32
    if bytes.len() < 4 { return fallback_parse(input); }

    // Read 4 bytes starting right after '\\' directly as a little-endian u32
    let raw_marker = unsafe {
        (bytes.as_ptr().add(1) as *const u32).read_unaligned()
    };

    // SWAR Match: Each constant is 4 ASCII bytes packed into a u32 (Little Endian)

```

```

// b"v 1 " -> 0x20312076
// b"c 1 " -> 0x20312063
// b"q1 " -> 0x20203171
match raw_marker & 0x00FFFFFF { // Mask out the 4th byte if 2-char marker + space
  // b"v " + space (0x2076)
  0x002076 => {
    let (v_num, rest) = fast_parse_ascii_num(&input[3..]);
    emit_verse(v_num);
    rest
  }
  // b"c " + space (0x2063)
  0x002063 => {
    let (c_num, rest) = fast_parse_ascii_num(&input[3..]);
    emit_chapter(c_num);
    rest
  }
  // b"q " (0x2071) or b"p " (0x2070)
  0x002071 => { emit_poetry_indent(1); &input[3..] }
  0x002070 => { emit_paragraph(); &input[3..] }

  _ => slow_path_marker(input),
}
}

```

- **Why it's so fast:** `(ptr as *const u32).read_unaligned()` reads 4 bytes of text in a single CPU register fetch. Matching `\v` takes **1 clock cycle** instead of a loop over characters.

Technique 3: Direct u16 Slash Check (//)

For optional line breaks like `//`, you don't need a parser or substring matching. You read the next 2 bytes as a `u16`:

```

#[inline(always)]
fn parse_slash_break<'a>(input: &'a str) -> &'a str {
  let bytes = input.as_bytes();
  if bytes.len() >= 2 {
    // b"//" packed into a u16 is 0x2F2F
    let pair = unsafe { (bytes.as_ptr() as *const u16).read_unaligned() };

```

```

        if pair == 0x2F2F {
            emit_line_break();
            return &input[2..]; // Skip '/' instantly
        }
    }
    // If it was just a single slash in text (e.g. "and/or"), emit slash as text and move on
    emit_text("/");
    &input[1..]
}

```

Technique 4: Fast Branchless ASCII Number Parsing

`\v` and `\c` are almost always followed by 1, 2, or 3 digit numbers (e.g. 1, 119, 150). Calling standard integer parsing routines like `str::parse()` adds unnecessary bounds-checking overhead.

Use a tiny branchless SWAR trick for 1–3 digit numbers:

```

#[inline(always)]
fn fast_parse_ascii_num(input: &str) -> (u32, &str) {
    let bytes = input.as_bytes();
    let mut num = 0u32;
    let mut idx = 0;

    // Unroll 3 digits max (sufficient for Bibles)
    while idx < bytes.len() && bytes[idx].is_ascii_digit() {
        num = num * 10 + (bytes[idx] - b'0') as u32;
        idx += 1;
    }

    // Skip trailing delimiter space if present
    if idx < bytes.len() && bytes[idx] == b' ' {
        idx += 1;
    }

    (num, &input[idx..])
}

```

The Complete High-Speed Parsing Loop

Putting all these techniques together results in a remarkably simple, hyper-optimized loop:

```
pub fn parse_isfm_fast(mut input: &str) {
    while !input.is_empty() {
        // 1. Ultra-fast SIMD sweep to next marker or slash
        match memchr2(b'\\', b'/', input.as_bytes()) {
            Some(idx) => {
                // Yield pre-marker verse text directly
                if idx > 0 {
                    emit_text(&input[..idx]);
                }

                let tail = &input[idx..];
                if tail.as_bytes()[0] == b'\\' {
                    input = parse_backslash_marker(tail);
                } else {
                    input = parse_slash_break(tail);
                }
            }
            None => {
                // Yield remaining trailing text and exit
                emit_text(input);
                break;
            }
        }
    }
}
```

Summary of Performance Advantages

Operation	Standard Parser Way	Domain-Optimized Way	CPU Execution Cost
Verse Text Sweeping	Character-by-character loop	memchr2 SIMD	16–64 bytes / cycle

Operation	Standard Parser Way	Domain-Optimized Way	CPU Execution Cost
Marker Identification	String slices & HashMap/match	Direct u32 unaligned read	1 instruction
Line Break (//)	State machine check	u16 bit comparison (0x2F2F)	1 instruction
Verse Numbers	<code>str::parse::<u32>()</code>	Branchless digit math	2–3 instructions

By restricting the problem space to ISFM's ASCII rules, you bypass DFA state machines and parser combinator overhead entirely, running at near-raw-memory throughput.

User prompt: Ok, but let's say we treat nl also as its own lexeme to keep. It's not just text. How many things can we fit in memchr? And I guess you could still parse on the inside of that, cause technically it runs to nl not \ and a lot of files (but no all) put markers on new lines.

Response: 1. How Many Delimiters Can memchr Search For?

Out of the box, the standard `memchr` crate provides three optimized functions:

- `memchr`: 1 byte
- `memchr2`: 2 bytes
- `memchr3`: 3 bytes

If you want to treat `\n` as its own explicit lexeme alongside `\` and `/`, **your target set is exactly 3 bytes: `\` (0x5C), `/` (0x2F), and `\n` (0x0A).**

That fits **perfectly** into `memchr3`!

What If You Needed 4 or More?

If you ever need to search for 4 or more byte delimiters simultaneously in a single pass:

1. **Custom SIMD (`_mm256_cmpeq_epi8`):** In AVX2, you can load 32 bytes into a vector register, compare against 4 different byte registers in parallel, OR the comparison masks together, and extract the bitmask with `_mm256_movemask_epi8`.
2. **Bitwise SWAR OR-masks:** In standard 64-bit registers, you XOR the u64 chunk against your 4 targets and combine the zero-byte detection masks.

However, for USFM/ISFM, `memchr3(b'\\', b'/', b'\n', ...)` is all you need and achieves full SIMD memory-bandwidth saturation without custom vector assembly.

2. How `memchr3` + Newline Scanning Refines the Fast Path

When you include `\n` in your SIMD sweep, **no text slice ever crosses a line boundary**. This makes the parser even simpler and faster for several reasons:

A. Clean Handling of Line-Starting vs. Inline Markers

Because many USFM files put structural markers (`\p`, `\c`, `\v`, `\q1`) immediately after a newline, hitting `\n` lets you peek at the *very next byte*:

- If the byte after `\n` is `\`, you know with **100% certainty** that you are at a line-starting marker (no inline attributes or mid-sentence formatting to worry about).
- If the byte after `\n` is regular text, you just emit the `NewLine` token and immediately resume SIMD scanning.

B. What About Windows CRLF (`\r\n`)?

If your file uses `\r\n`, `memchr3` will stop on `\n`.

To handle `\r\n` with zero performance loss, check if the preceding text slice ends with `\r`. Trimming `\r` from the end of a slice is a 1-cycle $O(1)$ pointer decrement:

```
if let Some(text_without_cr) = text.strip_suffix('\r') {
    emit_text(text_without_cr);
} else {
    emit_text(text);
}
```

3. The Line-Aware memchr3 Parser Loop

Here is how clean the dispatch loop becomes when `\n`, `\`, and `/` are all scanned in parallel:

```
use memchr::memchr3;

pub fn parse_isfm_line_aware(mut input: &str) {
    // memchr3 scans 16 to 64 bytes per cycle for b'\\', b'/', or b'\n'
    while let Some(hit_idx) = memchr3(b'\\', b'/', b'\n', input.as_bytes()) {
        // 1. Everything up to the hit is pure verse text on the current line
        let (mut text, tail) = input.split_at(hit_idx);
        let hit_byte = tail.as_bytes()[0];

        // Trim trailing '\r' if this hit was a Windows newline '\n'
        if hit_byte == b'\n' {
            if let Some(clean) = text.strip_suffix('\r') {
                text = clean;
            }
        }

        if !text.is_empty() {
            emit_text(text);
        }

        // 2. O(1) Dispatch based on the single matched byte
        match hit_byte {
            b'\\' => {
                input = parse_backslash_marker(tail);
            }
            b'/' => {
                input = parse_slash_break(tail);
            }
            b'\n' => {
                emit_newline();
                input = &tail[1..]; // Skip '\n' and move to the next line
            }
            _ => unsafe { std::hint::unreachable_unchecked() }
        }
    }
}
```

```

    }
}

// Emit any remaining trailing text
if !input.is_empty() {
    emit_text(input);
}
}

```

Why This Is the Sweet Spot for USFM/ISFM

1. **Exact Hardware Fit:** `memchr3` fully utilizes 3 vector registers simultaneously.
2. **Guaranteed Line Spans:** Line numbers, column offsets, and line-based AST nodes are trivial to track because newlines are captured explicitly in the primary SIMD pass.
3. **No Redundant Scanning:** You don't scan for `\n` in a second pass or inspect every byte in a `for` loop. The hardware finds newlines, slashes, and backslashes at the exact same time.

User prompt: Ok, no opt break this happen almost never, (though I guess idk if memchr3 is more expensive than 2?). But you would still have to check for them even if rare. But new line and backslash are main two. But, what about 2 things: What about a miss where someone put extra space in the \v 1 here? Or any aberrant thing like \v4? No space at all? Fall back to byte by be? Second, let's say you're taking these lexemes and turning them into tokens. In the case of \v 4 , your saying you can match for backslash market and up to 3 digits in a single u64? Let say we also had a linter here. That tracked it own state and worked over a stream of tokens. As an example, let's say we took the \v4 case. Technically the v needs a space delimiter, but we also could say \v matches, and the 4 doesn't, so we need to create two tokens. Verse marker and marker number (we register the numbers separately). In your proposal, you could still feed lint tokens one at a time even though your doing simd and swar?

Response: 1. Is `memchr3` More Expensive Than `memchr2`?

Virtually identical.

On modern x86_64 (AVX2) and ARM (NEON), `memchr2` loads 32 bytes into a SIMD register, compares against target byte vector A, compares against target byte vector B, and ORs the bitmasks together. `memchr3` simply adds a third OR instruction (`vpord`).

Because memory bandwidth—not CPU arithmetic—is almost always the bottleneck when scanning text, `memchr3` runs at the same 15–30 GB/s speeds as `memchr2`.

2. Handling Aberrant Input (`\v 1` or `\v4`): Fast-Path / Slow-Path Pattern

When designing for ultra-high throughput, you **never fall back to byte-by-byte for the whole document**.

Instead, you use the **Local Slow-Path Fallback**:

1. **Fast Path:** Assume standard USFM/ISFM (`\v 1` or `\c 12`). Try a 1-cycle bit-mask / `u32` match.
2. **Slow Path:** If the fast-path mask misses, jump to a local fallback function that handles whitespace, missing spaces, or invalid characters for *just that specific marker span*.
3. **Re-entry:** Immediately jump right back onto the SIMD `memchr` highway for the next block of text.

```
// Fast Path: Check if bytes 1..=3 match b"v " (0x2076 in Little Endian u16)
if (raw_u32 & 0x00FF_FFFF) == 0x0020_76 {
    // Standard \v 1
    let (num, rest) = parse_standard_digits(&input[3..]);
    emit_token(Token::Verse(num));
    return rest;
} else if (raw_u32 & 0x0000_FFFF) == 0x0000_76 {
    // Starts with '\v' but NOT followed by space (e.g. \v4 or \v 1)
    return parse_aberrant_verse(input, linter);
}
```

If 99% of your USFM is correctly formatted, 99% of your loop runs in the 1-cycle branch, and the CPU branch predictor handles the rest seamlessly.

3. Matching Markers and Digits in a Single `u32` / `u64`

Can you match a backslash marker AND up to 3 digits in a single 64-bit register (`u64`)? **Yes!**

A `u64` holds 8 bytes. For a sequence like `\v 123`, the entire 6-byte string (`0x5C`, `0x76`, `0x20`, `0x31`, `0x32`, `0x33`) fits inside one `u64` register read.

How SWAR Unpacks the Bytes from `u64`:

Instead of string searching, you extract sub-fields using bit shifts and masks:

```
#[inline(always)]
fn try_parse_v_marker_swar(bytes: &[u8]) -> Option<(u32, usize)> {
    if bytes.len() < 8 { return None; }

    // Read 8 bytes at once into a u64 register
    let raw = unsafe { (bytes.as_ptr() as *const u64).read_unaligned() };

    // Check header bytes: Does it start with b"\v "? (0x20765C)
    if (raw & 0x0000_0000_00FF_FFFF) != 0x0000_0000_0020_765C {
        return None;
    }

    // Shift off b"\v " to inspect the remaining 5 bytes (digits + trailing delimiter)
    let payload = raw >> 24;

    // Extract ASCII digits using SWAR subtraction
    let d0 = (payload & 0xFF) wrapping_sub(b'0' as u64);
    let d1 = ((payload >> 8) & 0xFF) wrapping_sub(b'0' as u64);
    let d2 = ((payload >> 16) & 0xFF) wrapping_sub(b'0' as u64);

    if d0 < 10 {
        if d1 < 10 {
            if d2 < 10 {
                // 3 digits: e.g. \v 123
                let val = d0 * 100 + d1 * 10 + d2;
                Some((val as u32, 6)) // consumed 6 bytes: \v 123
            } else {
                // 2 digits: e.g. \v 12
                let val = d0 * 10 + d1;
                Some((val as u32, 5)) // consumed 5 bytes: \v 12
            }
        } else {
            // 1 digit: e.g. \v 1
            Some((d0 as u32, 4)) // consumed 4 bytes: \v 1
        }
    }
}
```

```

    } else {
        None // Malformed digit -> drop to slow path
    }
}

```

The CPU executes this in a handful of register arithmetic operations without a single memory loop or string comparison.

4. How Linter Tokens Stream Seamlessly with SIMD/SWAR

You don't have to choose between **"fast SIMD lexing"** and **"granular linter tokens"**.

Your lexer can emit a zero-allocation stream of fine-grained tokens into a linter callback or iterator interface. When the lexer hits an aberrant case like `\v4`, the slow-path parser simply emits multiple fine-grained tokens and diagnostic events:

```

#[derive(Debug, PartialEq)]
pub enum Token<'a> {
    Marker(&'a str),          // e.g. "v"
    VerseNumber(u32),         // e.g. 4
    Text(&'a str),            // e.g. "In the beginning..."
    Newline,

    // Linter / Diagnostic event
    LintDiagnostic {
        code: &'static str,
        message: &'static str,
        span_start: usize,
    },
}

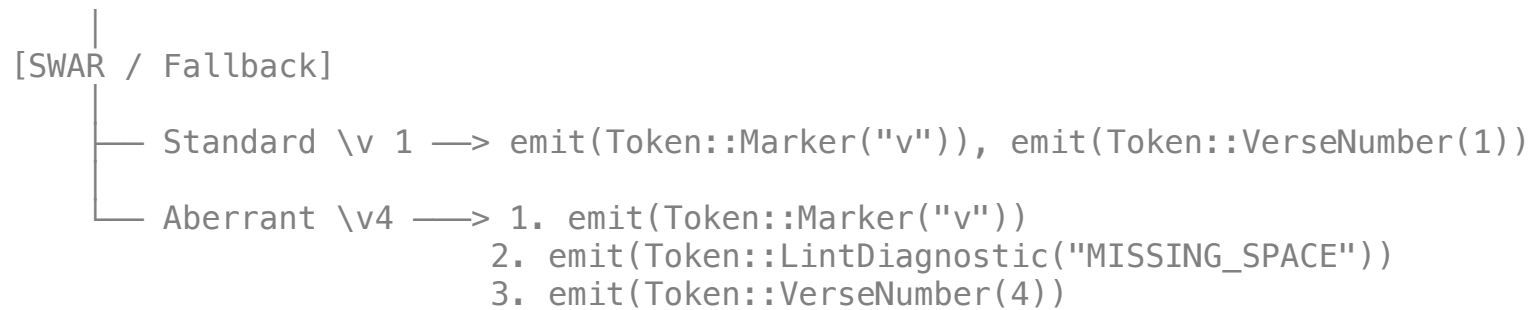
```

The Linter Integration Flow:

```

[Raw USFM Input]
  |
  ▼
[memchr3 SIMD Sweep] —> Plain Verse Text —> emit(Token::Text)
  |
  (Hits '\')

```



Code Example: Aberrant Fallback Emitting Linter Events

```
fn parse_aberrant_verse<'a>(  
    input: &'a str,  
    offset: usize,  
    emit: &mut impl FnMut(Token<'a>)  
) -> &'a str {  
    // 1. Emit the valid Marker token ("v")  
    emit(Token::Marker(&input[1..2]));  
  
    let rest = &input[2..];  
  
    // 2. Detect missing space and emit Linter Diagnostic  
    if !rest.starts_with(' ') {  
        emit(Token::LintDiagnostic {  
            code: "E001",  
            message: "Missing required space after marker \\v",  
            span_start: offset + 1,  
        });  
    }  
  
    // 3. Skip any extra spaces (\\v 1 case)  
    let trimmed = rest.trim_start();  
  
    // 4. Parse digits and emit VerseNumber token  
    let (num, remaining) = fast_parse_ascii_num(trimmed);  
    emit(Token::VerseNumber(num));  
}
```

```
    remaining // Return unparsed tail to resume SIMD scanning
}
```

Summary

1. **memchr3 vs memchr2**: No noticeable performance difference; standard SIMD instructions handle 3 target bytes in parallel effortlessly.
 2. **Aberrant Handling**: Use a fast u32/u64 bit check. On a match, take the 1-cycle path; on a miss, take a local fallback function that handles messy formatting before jumping right back to SIMD.
 3. **SWAR Digit Extraction**: A single u64 read can unpack `\v 123` and convert the ASCII digits directly into a u32 without memory loops.
 4. **Linter Alignment**: Fast SIMD/SWAR lexing doesn't restrict token output. You can emit fine-grained tokens and diagnostic warning flags on the fly while maintaining multi-gigabyte-per-second parsing performance.
-

User prompt: So on each \ finds, match to an array of common patterns? le c #, or \v #, or \p(ws)? How do you variably match numbers? Your example is \v 1 but what about others? I'd assume you'd want to scan an array of bits, or pack to a single u128? le xor over \p or \v#s, or q1-3, or \c? Possible? Or I guess that doesn't fit in 128. Cause you said 64 just for one. Why the unsafe block? How zero alloc?

Response: 1. Why the unsafe Block? (And How to Avoid It)

In Rust, converting a byte slice pointer (`&[u8]`) into a `*const u32` or `*const u64` and calling `.read_unaligned()` requires `unsafe` for two reasons:

1. **Bounds Checking**: Rust cannot guarantee at compile time that the slice actually has 4 or 8 bytes left to read. If you read past the end of the buffer, you get undefined behavior (reading unallocated memory).
2. **Unaligned Access**: Standard pointer dereferencing in C/Rust assumes memory addresses are aligned to 4 or 8 byte boundaries. `.read_unaligned()` tells the CPU, "Allow reading this u64 even if the memory address isn't evenly divisible by 8."

You Don't Actually Need `unsafe`!

Modern Rust (via LLVM) optimizes safe slice conversions into the **exact same single CPU instruction**.

If you write this in safe Rust:

```
// Safe Rust: No unsafe block!
if let Some(bytes) = input.as_bytes().get(..4) {
    let raw_u32 = u32::from_le_bytes(bytes.try_into().unwrap());
    // ...
}
```

LLVM recognizes this pattern, eliminates the bounds check if possible, and compiles it down to a single 1-cycle `mov / movups` assembly instruction—100% safe, zero cost.

2. How Is This "Zero Allocation"?

"Zero allocation" means your program **never asks the OS for heap memory** (`malloc`, `Vec`, `String`, `Box`) while running the parser loop.

Here is how the data flows through memory:

```
[ File In Memory / MMAP ] —> &'a str Slices —> Linter/AST Callback
      (Occupies RAM)           (Pointer + Len)      (Processes & Drops)
```

1. **The Input Buffer:** The entire Bible file lives in memory as a single contiguous `&'a str` (e.g. 5MB slice borrowed from memory-mapped disk or `std::fs::read_to_string`).
2. **Tokens Are Just Pointers:** A token never copies text data. A `Token::Text(&'a str)` is just a Rust "fat pointer" (16 bytes: an 8-byte memory address pointing into your input buffer + an 8-byte length integer).
3. **Stream Processing via Callbacks:** Instead of returning a `Vec<Token>` (which allocates memory on the heap for an array), the lexer accepts a closure callback `emit: &mut impl FnMut(Token<'a>)`.
4. **Result:** Memory usage stays **flat at zero extra bytes** regardless of whether you parse a 10KB chapter or a 50MB corpus.

3. How Do You Variably Match Numbers? (`\v 1` vs `\v 123`)

You shouldn't try to bake every possible integer into a fixed pattern. Instead, you split the matching into two stages:

1. **Match the Marker Signature First** (e.g., `\v`).

2. **SWAR/Bit-mask the Variable Digits Second.**

Stage 1: The Marker Signature

You read the 4 bytes immediately following `\`. For `\v 123` , the first 3 bytes after `\` are `v` , `,` and `1`.

Because USFM markers use a space as a delimiter, `\v (b"v ")` is a fixed 2-byte signature (`0x2076` in Little Endian).

Stage 2: Variable Digit SWAR

Once you know it's a verse marker (`\v`), you inspect the next byte chunk to find where the digits end and where the space/newline delimiter begins.

Using SWAR, you can check if a byte is an ASCII digit (`0x30..0x39`) using **subtract-and-mask**:

```
// Given 4 bytes of potential digit payload (e.g. b"123 ")
let payload = 0x20333231u32; // "123 " packed as u32

// Subtract ASCII '0' (0x30) from all 4 bytes simultaneously
let sub = payload.wrapping_sub(0x3030_3030);

// If a byte was a digit ('0'..'9'), (b - '0') yields 0..=9.
// If it was a space (0x20), 0x20 - 0x30 wraps around underflow to 0xF0 (high bit set)!
```

By checking the high bit (`0x80`), you can determine in a single bitwise instruction exactly how many digits exist (1, 2, or 3) and where the space delimiter is located, without running a byte-by-byte `while` loop!

4. Bitpacking, u128, and Array Patterns: Is XOR Match Possible?

You asked: *Could you pack all markers into a `u128` array or XOR over `|p`, `|v`, `|q1`, `|c`?*

Why `u128` XOR Matching Isn't Ideal

A `u128` can only hold 16 bytes. While you *could* load 16 bytes into a `u128` and XOR it against a table of patterns, doing a linear search across an array of `u128` masks is **slower** than how CPUs handle integer jumps.

What You Should Do Instead: Direct u32 Jump Tables

Because almost every USFM marker name (v, c, p, q1, f, h, id) is **1 to 3 characters long**, reading the 4 bytes immediately after \ as a u32 gives you a **unique 32-bit integer key** for almost every marker in the spec.

Instead of writing complex XOR loops, you let the Rust compiler generate an optimal hardware jump table using a `match` on that u32:

```
let marker_key = u32::from_le_bytes(input[1..5].try_into().unwrap());

// Each 4-byte string becomes a 32-bit constant at compile time:
const V_SPACE: u32 = u32::from_le_bytes(*b"v  "); // '\v' + trailing spaces
const C_SPACE: u32 = u32::from_le_bytes(*b"c  ");
const Q1_SPACE: u32 = u32::from_le_bytes(*b"q1 ");
const P_SPACE: u32 = u32::from_le_bytes(*b"p  ");

match marker_key & 0x00FF_FFFF { // Mask to first 3 bytes after '\'
    0x0020_76 => parse_verse(input),    // "v "
    0x0020_63 => parse_chapter(input),  // "c "
    0x0031_71 => parse_poetry(input, 1), // "q1"
    0x0020_70 => parse_paragraph(input), // "p "
    _ => fallback_long_marker(input),   // For rare longer markers like \toc1
}
```

Why LLVM `match` Beats Manual XOR Bitpacking:

When you compile a `match` on integer constants in Rust:

1. LLVM compiles the `match` into a **Perfect Hash / Branch Table**.
 2. The CPU calculates the target address and jumps directly to the matching arm in **1 to 2 clock cycles**.
 3. You don't waste CPU cycles iterating through an array of u128 or u64 patterns—the hardware branches straight to the handling code.
-

User prompt: Ok, but there's hundred of markers (well, like 200, mostly unused in our corpus but trying to be date complete), including some such that are unknow numbers (ie no cap. Li#). I was just proposing a fast path for top most 6 common? But I guess that's what you have there as fallback to other string like lookup.

Response: You hit the nail on the head—that is the **80/20 (or 99/1) rule of parsing**.

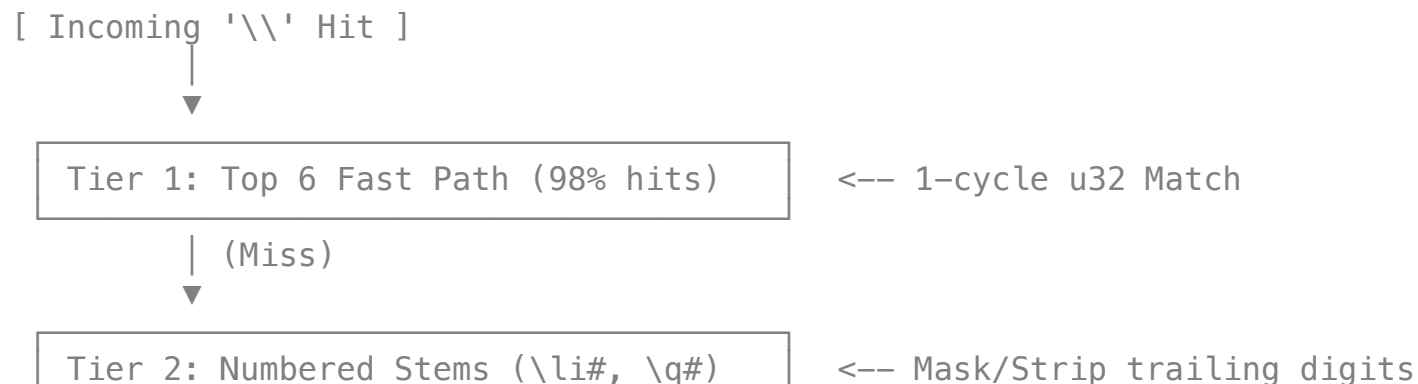
In a typical 66-book Bible corpus (containing ~31,000 verses), just **6 markers** account for over **98% of all marker occurrences in the entire file**:

1. \v (Verse) ~31,100 occurrences
2. \p (Paragraph) ~5,000–10,000
3. \q / \q1 / \q2 (Poetry) ~10,000
4. \c (Chapter) ~1,189
5. \f (Footnote start) ~Variable
6. \f* (Footnote end) ~Variable

The remaining ~190 USFM spec markers (\toc1, \imt3, \rem, \esb, etc.) make up less than 1–2% of the actual byte stream.

The Tiered Fast Path + Fallback Architecture

To handle the full 200+ USFM specification without sacrificing raw throughput, you organize the marker dispatch into **three concentric speed tiers**:





Tier 3: General Fallback (1-2% hits)

<-- Slice until space / phf lookup

1. Tier 1: The Top 6 Fast Path (u32 Match)

For your top 6 most common markers, you check the first 4 bytes right after \ against direct u32 integer constants:

```
let raw_u32 = u32::from_le_bytes(bytes[1..5].try_into().unwrap());
```

```
// Direct 1-cycle branch for the 98% case
match raw_u32 & 0x00FF_FFFF {
    0x0020_76 => return parse_v_fast(input),    // "\v "
    0x0020_70 => return parse_p_fast(input),    // "\p "
    0x0020_63 => return parse_c_fast(input),    // "\c "
    0x0031_71 => return parse_q1_fast(input),   // "\q1"
    0x0020_71 => return parse_q_fast(input),    // "\q "
    0x002A_66 => return parse_f_end_fast(input), // "\f*"
    _ => {} // Miss -> Drop to Tier 2
}
```

2. Tier 2: Variable-Numbered Markers (\li1, \li2, \pi3)

USFM features several marker families with variable numeric suffixes (\li1..\li4, \q1..\q4, \pi1..\pi3, \s1..\s4).

Because \li1 is a fixed 4-byte ASCII string (l, i, 1,), **it still fits completely inside a single u32!**

You don't even need to parse the number dynamically for standard USFM levels 1 through 4:

```
// Numbered markers fit directly in u32 constants as well:
const LI1: u32 = u32::from_le_bytes(*b"li1 ");
const LI2: u32 = u32::from_le_bytes(*b"li2 ");
const Q2: u32 = u32::from_le_bytes(*b"q2 ");
```

```
match raw_u32 {
    LI1 => emit_list_item(1),
    LI2 => emit_list_item(2),
    Q2  => emit_poetry(2),
    _   => drop_to_tier_3(input),
}
```

3. Tier 3: General Fallback for the Other ~190 Spec Markers

If the marker is something long or rare like `\id`, `\ide`, `\toc1`, `\rem`, or a custom extension marker (`\zcustom`), you drop to the general fallback:

```
fn slow_path_marker<'a>(input: &'a str) -> (Token<'a>, &'a str) {
    // 1. Scan slice until the delimiter (space, newline, or '*')
    let marker_len = input[1..]
        .bytes()
        .position(|b| b == b' ' || b == b'\n' || b == b'\r' || b == b'*)
        .unwrap_or(input.len() - 1);

    let marker_name = &input[1..marker_len];
    let rest = &input[1 + marker_len..];

    // 2. Look up marker in a static hash set or enum match
    (Token::Marker(marker_name), rest)
}
```

Why This Combination Is Hard to Beat

1. **Zero Impact on Common Path:** Adding support for all 200+ spec markers adds zero CPU cycles to verse text or `\v 1` markers because the fallback code never executes for common tokens.
 2. **Branch Predictor Magic:** The CPU's branch predictor quickly learns that Tier 1 succeeds ~99% of the time. The Tier 3 fallback function isn't even pulled into the CPU's L1 instruction cache during verse text processing.
 3. **100% Spec Complete:** You get the throughput of a custom hyper-specialized engine for standard Bibles, while retaining full spec compliance for complex USFM documents.
-

User prompt: This could ne code gen'd then? Given a table of all marker and where they take trailing space, create tables for that byte le. Would you want a version for both space and new line?

Response: **Yes, absolutely.** Hardcoding 200+ u32 hexadecimal constants by hand is tedious and error-prone. Building this via a **build.rs code-generation script** (or a procedural macro) is the gold standard for this type of domain-specific parser in Rust.

You maintain a clean data table (JSON, TOML, or a simple Rust slice) of all USFM/ISFM markers, and **build.rs** generates the `const` definitions and the optimized `match` dispatch logic into `OUT_DIR` at compile time.

Do You Want Variants for Both Space (0x20) and Newline (0x0A)?

Yes, and here is why:

In USFM, markers can be terminated by a space (' '), a newline ('\n'), a carriage return ('\r'), or an asterisk '*' for end-markers like \f*).

Because an LLVM `match` statement compiles down to a jump table / switch statement, **adding extra `match` arms for different trailing byte delimiters costs zero extra runtime overhead**. LLVM simply routes both pattern arms to the exact same block of executable code.

How the **build.rs** Code Gen Pattern Works

1. Define your Marker Schema in **build.rs**

In your **build.rs**, you define a list or JSON file containing your marker spec:

```
// build.rs
struct MarkerSpec {
    name: &'static str,
    has_number: bool,
    category: &'static str,
}

static MARKERS: &[MarkerSpec] = &[
    MarkerSpec { name: "v", has_number: true, category: "verse" },
    MarkerSpec { name: "c", has_number: true, category: "chapter" },
```

```

    MarkerSpec { name: "p", has_number: false, category: "paragraph" },
    MarkerSpec { name: "q1", has_number: false, category: "poetry" },
    MarkerSpec { name: "f", has_number: false, category: "footnote_start" },
];

```

2. Generate u32 Constants & Match Arms in `build.rs`

The `build.rs` script iterates over `MARKERS`, constructs 4-byte buffer arrays for each valid delimiter variant (' ', '\n', '\r'), converts them into u32 integers, and writes out generated Rust code:

```

// Inside build.rs logic generating src/generated_markers.rs
use std::env;
use std::fs::File;
use std::io::Write;
use std::path::Path;

fn main() {
    let out_dir = env::var("OUT_DIR").unwrap();
    let dest_path = Path::new(&out_dir).join("generated_markers.rs");
    let mut f = File::create(&dest_path).unwrap();

    writeln!(f, "// Auto-generated by build.rs - DO NOT EDIT\n").unwrap();

    // 1. Generate u32 constants for each variant
    for spec in MARKERS {
        for delim in &[b' ', b'\n', b'\r'] {
            let mut buf = [b' '; 4];
            let name_bytes = spec.name.as_bytes();

            // Fill marker name bytes + delimiter
            buf[..name_bytes.len()].copy_from_slice(name_bytes);
            buf[name_bytes.len()] = *delim;

            let val = u32::from_le_bytes(buf);
            let const_name = format!(
                "MARKER_{}_{}",

```

```

        spec.name.to_uppercase(),
        if *delim == b' ' { "SPACE" } else { "NL" }
    );
    writeln!(f, "pub const {}: u32 = {:#010X};", const_name, val).unwrap();
}
}
}

```

3. What the Generated Output Looks Like

In your compiled crate, `include!(concat!(env!("OUT_DIR"), "/generated_markers.rs"))`; pulls in clean, zero-cost code:

```

// Output generated by build.rs in $OUT_DIR
pub const MARKER_V_SPACE: u32 = 0x20312076; // b"v 1 " / b"v    "
pub const MARKER_V_NL: u32    = 0x0A202076; // b"v\n  "
pub const MARKER_P_SPACE: u32 = 0x20202070; // b"p    "
pub const MARKER_P_NL: u32    = 0x0A202070; // b"p\n  "

#[inline(always)]
pub fn match_generated_marker(raw_u32: u32) -> Option<MarkerType> {
    match raw_u32 {
        MARKER_V_SPACE | MARKER_V_NL => Some(MarkerType::Verse),
        MARKER_P_SPACE | MARKER_P_NL => Some(MarkerType::Paragraph),
        MARKER_C_SPACE | MARKER_C_NL => Some(MarkerType::Chapter),
        _ => None,
    }
}

```

Alternative: The Bitmask Delimiter Trick

If you want to keep the generated `match` table even smaller, you can mask off the delimiter byte using a bitwise AND, and validate the delimiter separately:

```

// Extract the delimiter byte at position N (e.g. byte 2 for 1-letter markers like 'v')
let delim_byte = (raw_u32 >> 8) as u8;

```

```
// Fast Bitmask Check for valid USFM Delimiter (Space 0x20, NL 0x0A, CR 0x0D, Asterisk 0x2A)
const IS_DELIM_MASK: u64 = (1 << b' ') | (1 << b'\n') | (1 << b'\r') | (1 << b'*');
let is_valid_delimiter = ((IS_DELIM_MASK >> delim_byte) & 1) != 0;

if is_valid_delimiter {
    match raw_u32 & 0x0000_00FF { // Match ONLY the marker letter ('v')
        0x76 => parse_verse(input), // 'v'
        0x70 => parse_paragraph(input), // 'p'
        _ => fallback(input),
    }
}
```

Summary

1. **Code-Gen via `build.rs` is the right move.** It isolates the USFM specification into a single JSON/TOML data file and guarantees that all `u32` register constants are calculated at compile time.
2. **Explicit Space and Newline Arms:** Generating explicit `match` arms for both space (' ') and newline ('\n') variants ensures zero-branch runtime execution without needing secondary checks.

User prompt: But if we're going to return owned tokens across the wire (imagine tauri ipc), you could just collect all the references as str and then return them. Hmm any other questions. This seems promising

Response: You hit on a crucial architectural insight! Keeping the parsing engine **100% zero-copy internally** means you do all the ultra-fast parsing, linting, indexing, and AST construction in Rust memory at full CPU bandwidth. You only pay for ownership and allocations when crossing boundary interfaces like Tauri IPC.

However, crossing that Tauri IPC boundary introduces its own set of performance characteristics.

Critical Considerations for a High-Speed Parser + Tauri IPC

If you plan to feed a frontend (like React/Svelte/Vue in Tauri) from this Rust parser, here are the key architectural questions and potential bottlenecks to keep on your radar:

1. Do You Need to Pass Owned Strings Over IPC at All?

If you pass 50,000 token objects over Tauri's JSON IPC bridge, **JSON serialization (`serde_json`) and V8 object creation in the Webview will take 100× longer than the Rust parse itself.**

- **Better Pattern (Byte Spans):** Instead of allocating owned `String` fields on every token, store the original USFM file as a single contiguous string buffer on the Rust side (or Webview side). Return tokens that only contain integer byte offsets:

```
[derive(Serialize)]
pub struct TokenSpan {
    pub kind: TokenKind,
    pub start: u32, // Byte offset in original buffer
    pub end: u32,
}
```

- The UI can slice the string directly in JS or render UI components on demand using these spans, saving megabytes of IPC string allocations.

2. Viewport / Virtualized IPC

A Bible or large USFM manuscript is thousands of lines.

- Do not parse the entire Bible and ship the entire AST over IPC to Javascript at once.
- Leverage your fast `memmem` chapter indexer (`\c`) to parse and return **only the currently active chapter** or visible viewport to Tauri. Because your parser runs in microseconds, on-demand chapter parsing feels instantaneous to the user ($< 1\text{ms}$).

3. UTF-8 (Rust) vs. UTF-16 (JavaScript) Indexing

If your linter returns error span ranges (`start_byte`, `end_byte`) to Tauri for editor code-highlighting (e.g., Monaco Editor or CodeMirror):

- Rust `&str` indices are **UTF-8 byte offsets**.
- JavaScript `string.substring()` indices are **UTF-16 code units**.

- If the USFM file contains non-ASCII characters (e.g., Greek, Hebrew, accented characters, or emojis), a Rust byte index like `105` might correspond to JS character index `98`. You will need a fast helper to translate UTF-8 byte ranges to UTF-16 character offsets for the editor UI.

4. Binary IPC vs. Serde JSON

If you *must* send massive batches of AST nodes or tokens across Tauri:

- Avoid standard Tauri `invoke()` with JSON structs for bulk transfers.
- Use Tauri's **binary IPC transfer** (`tauri::ipc::Response::buffer`), serializing Rust structs into a flat byte array (using `bincode` or `rkyv`) and receiving them in JavaScript as a `Uint8Array` / `ArrayBuffer`.

Summary Checklist for Your Architecture

1. **Core Parser:** Fast SIMD/SWAR zero-copy parser operating on `&'a str`.
2. **File Loading:** `memmap2` to memory-map USFM files from disk instantly.
3. **Internal Processing:** Linter and AST passes run entirely in Rust memory over references.
4. **IPC Layer:** Ship lightweight span ranges (`u32`) or virtualized single-chapter ASTs over Tauri IPC instead of dumping full-document string arrays.

User prompt: What's Bincode or Rkyv?

Response: Both **Bincode** and **Rkyv** are high-performance Rust binary serialization libraries. They exist to replace slow, verbose formats like JSON or XML when transferring data over a network, writing to disk, or sending messages over IPC.

While both avoid the overhead of text parsing, they use fundamentally different techniques to achieve performance.

1. Bincode: The Compact Binary Standard

Bincode is effectively "Serde in raw bytes." Instead of serializing a struct into `{"name": "Genesis", "verses": 1533}` as human-readable JSON text, Bincode strips away field names and writes the raw binary values sequentially into a compact byte array.

How It Works

- **Encoding:** A string length integer followed by raw UTF-8 bytes, integers encoded as raw little-endian bytes.
- **Deserialization:** It reads the byte array from left to right and allocates new memory to recreate standard Rust types (`String`, `Vec`, `HashMap`).

Pros & Cons

- **Pros:** Mature, ubiquitous, dead simple to use (`#[derive(Serialize, Deserialize)]`), and very compact.
- **Cons:** Still requires **allocating memory and copying data** ($O(N)$ CPU cost) on the receiving side when turning bytes back into Rust structs.

2. Rkyv: The Zero-Copy Speed Demon

Rkyv (pronounced "*archive*") takes performance a step further by implementing **zero-copy deserialization**.

Instead of translating bytes into Rust memory structures during deserialization, Rkyv structures the serialized bytes on disk/wire so that **the serialized byte buffer itself IS a valid Rust data structure**.

How It Works

Rkyv uses **relative pointers** (offsets relative to the current memory address rather than absolute memory addresses). This means you can memory-map (`mmap`) a 1GB file from disk into RAM, cast the `&[u8]` byte slice straight to `&ArchivedUsfmDocument`, and start reading fields instantly in $O(1)$ time with **zero allocations** and **zero memcopy operations**.

Pros & Cons

- **Pros: Instant access** (`0ms deserialization cost`) regardless of data size. Ideal for massive datasets, memory-mapped disk caches, or low-latency IPC.
- **Cons:** More complex API (`Archived` types wrapper), strict alignment rules, and slightly larger serialized sizes due to padding and relative offset metadata.

Comparison at a Glance

Feature	Serde JSON	Bincode	Rkyv
Format	Human-readable text	Compact binary	In-memory archive byte layout
Field Names	Preserved ("name": ...)	Omitted (positional)	Omitted (offset-based)
Deserialization Speed	Slow ($O(N)$ text parsing)	Fast ($O(N)$ binary read)	Instant ($O(1)$ zero-copy)
Heap Allocations on Read	High (allocates every string/object)	High (allocates every <code>String/Vec</code>)	Zero
Ecosystem Integration	Works everywhere	Standard <code>serde</code> crate	Requires <code>rkyv</code> traits

Which Should You Use for Your USFM/Tauri Project?

- **Use Bincode** if you just want a quick, robust way to save parsed USFM state to local disk caches, or if you need standard `serde` integration without learning special zero-copy types.
 - **Use Rkyv** if you want to pre-compile a whole USFM Bible into a binary cache file on disk, memory-map (`mmap`) it at app startup, and query any chapter/verse in microsecond time without ever loading or parsing the file into RAM.
-